8:00 PM Onwards

Wednesday, 1st April

HOST

Ben Burtenshaw

Community Education in AI at Hugging Face

Pulkit Aneja

Scaler Instructor

Bootcamp starts in : 00D 21H 28M 50S

Join Live

PROBLEM STATEMENT

THE TASK

Build a complete, real-world OpenEnv environment that an AI agent can learn from through the standard `step()` / `reset()` / `state()` API.

KEY REQUIREMENTS AT A GLANCE

- Must simulate a real-world task (not games or toys)
- Implement full OpenEnv spec: typed models, step()/reset()/state(), openenv.yaml

Hi! What can I help you with?

→

How to Submit?

- Deploy to Hugging Face Spaces + working Dockerfile
- README with environment description, action/observation spaces, setup instructions

Detailed Requirements

FUNCTIONAL REQUIREMENTS

Real-world task simulation

The environment must simulate a task humans actually do. Not games, not toys. Examples: email triage, code review, data cleaning, scheduling, customer support, content moderation.

OpenEnv spec compliance

Implement the full OpenEnv interface: typed Observation, Action, and Reward Pydantic models. step(action) → returns observation, reward, done, info. reset() → returns initial observation. state() → returns current state. openenv.yaml with metadata. Tested via openenv validate.

Minimum 3 tasks with agent graders

Each task defines a concrete objective an agent must accomplish, with a programmatic grader that scores performance (0.0–1.0). Tasks should range: easy → medium → hard. Graders must have clear, deterministic success/failure criteria.

Meaningful reward function

Provides signal over the full trajectory (not just binary end-of-episode). Rewards partial progress toward task completion. Penalizes clearly undesirable behavior (e.g. infinite loops, destructive actions).

Hi! What can I help you with?

→

How to Submit?

NON-FUNCTIONAL REQUIREMENTS

Deploys to a Hugging Face Space

Environment must run as a containerized HF Space tagged with openenv.

Containerized execution

Must include a working Dockerfile. The environment should start cleanly with docker build + docker run.

Documentation

README must include: environment description and motivation, action and observation space definitions, task descriptions with expected difficulty, setup and usage instructions, baseline scores.

Evaluation Criteria

PARAMETER	WEIGHT	DESCRIPTION
Real-world utility	30%	Does the environment m this to train or evaluate a
Task & grader quality	25%	Are tasks well-defined w measure success? Mean
Environment design	20%	Clean state managemen shaping, proper episode
Code quality & spec compliance	15%	Follows OpenEnv spec, c tested, Dockerfile works.

Hi! What can I help you with?

→

How to Submit?

- 0–5: Toy/artificial problem with no practical application
- 6–15: Valid domain but shallow modeling of the real task
- 16–25: Good domain modeling, would be useful for agent evaluation
- 26–30: Excellent — fills a real gap, immediate value for the RL/agent community

- 3+ tasks with difficulty range?
- Graders produce scores between 0.0–1.0?
- Graders deterministic and reproducible?
- Hard task genuinely challenges frontier models?

- reset() produces clean state?
- Action/observation types well-designed and documented?
- Reward function provides useful varying signal (not just sparse)?
- Episode boundaries sensible?

- openenv validate passes?
- docker build && docker run works?
- HF Space deploys and responds?
- Baseline script runs and reproduces scores?

- Domain we haven't seen in OpenEnv before?
- Reward design has interesting properties?
- Clever mechanics that make the environment engaging?

Hi! What can I help you with?

→

How to Submit?

Pass/fail gate — HF Space deploys, OpenEnv spec compliance, Dockerfile builds, basel

Scored — baseline agent re-run, standard Open LLM agent (e.g. Nemotron 3 Super) run

Top submissions reviewed by Meta and Hugging Face engineers for real-world utility, cr

DISQUALIFICATION CRITERIA

- Environment does not deploy or respond
- Plagiarized or trivially modified existing environments
- Graders that always return the same score
- No baseline inference script

Pre-Submission Checklist — all must pass or you're disqualified

HF Space deploys	Automated ping to the Space URL — must return 200 and respond to reset()
OpenEnv spec compliance	Validate openenv.yaml, typed models, step()/reset()/state() endpoints
Dockerfile builds	Automated docker build on the submitted repo
Baseline reproduces	Run the submitted inference script — must complete without error and produce scores
3+ tasks	

Hi! What can I help you with?

→

How to Submit?

MODEL_NAME The model identifier to use for inference.

HF_TOKEN Your Hugging Face / API key.

The inference script must be named `inference.py` and placed in the root directory of the project

Participants must use OpenAI Client for all LLM calls using above variables

Infra Restrictions: Runtime of inference script should be less than 20min

Make sure your env and inference can run on a machine with vcpu=2, memory=8gb

Validator: Run the pre-submission validation script before submitting

```
"""
Inference Script Example
===================================
MANDATORY
- Before submitting, ensure the following variables are defined in you
    API_BASE_URL   The API endpoint for the LLM.
    MODEL_NAME     The model identifier to use for inference.
    HF_TOKEN       Your Hugging Face / API key.

- The inference script must be named `inference.py` and placed in the
- Participants must use OpenAI Client for all LLM calls using above va
"""

import os
import re
import base64
import textwrap
from io import BytesIO
from typing import List, Optional, Dict

from openai import OpenAI
import numpy as np
```

Hi! What can I help you with?

→

How to Submit?

```
MODEL_NAME = os.getenv("MODEL_NAME")
MAX_STEPS = 8
MAX_DOM_CHARS = 3500
TEMPERATURE = 0.2
MAX_TOKENS = 200
FALLBACK_ACTION = "noop()"

DEBUG = True
```

```
#!/usr/bin/env bash
#
# validate-submission.sh — OpenEnv Submission Validator
#
# Checks that your HF Space is live, Docker image builds, and openenv
#
# Prerequisites:
#   - Docker:       https://docs.docker.com/get-docker/
#   - openenv-core: pip install openenv-core
#   - curl (usually pre-installed)
#
# Run:
#   curl -fsSL https://raw.githubusercontent.com/<owner>/<repo>/main/s
#
#   Or download and run locally:
#     chmod +x validate-submission.sh
#     ./validate-submission.sh <ping_url> [repo_dir]
#
# Arguments:
#   ping_url   Your HuggingFace Space URL (e.g. https://your-space.hf
#   repo_dir   Path to your repo (default: current directory)
#
# Examples:
#   ./validate-submission.sh https://my-team.hf.space
#   ./validate-submission.sh https://my-team.hf.space ./my-repo
```

Hi! What can I help you with?

→

How to Submit?

```
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BOLD='\033[1m'
NC='\033[0m'
```

Submission window opens on 28th March

Deadline: 8 Apr 11:59 PM

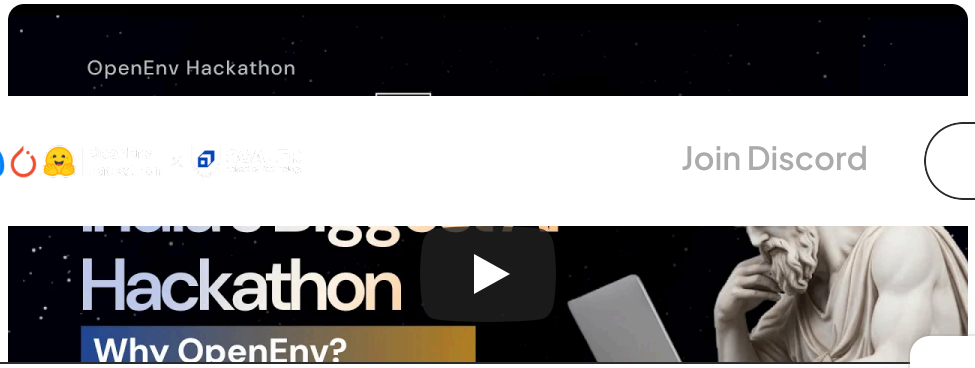
Submit your Assessment →

STUDY MATERIAL

4 modules · ~3.5 hours

Each module: read the README first, then open the notebook in Colab. No local setup needed.

ESSENTIAL FOR ROUND 1 45 min



What you'll do

Connect to 3 real

Join Discord Help Log out

an Echo bot, a Catch game, and

Hi! What can I help you with?

→

How to Submit?

Read Concept

ESSENTIAL FOR ROUND 1 50 min

ESSENTIAL FOR ROUND 1 45 min

MOST IMPORTANT FOR ROUN

View full course repository

GUIDE

What to Expect Prerequisites How to Submit

Python 3.10+

Install 3.10, 3.11, or 3.12.

```
$ python --version
```

Copy

Hi! What can I help you with?

→

How to Submit?

Hugging Face CLI

Deploy to HF Spaces.

```
$ pip install huggingfac
```
Copy

```
$ huggingface-cli login
```
Copy

OpenEnv

The framework.

```
$ pip install openenv-core
```
Copy

Google Colab

Prep course runs in Colab. Free tier works.

```
$ pip install openenv-core
```
Copy

OpenEnv

The framework.

```
→ colab.research.google.com
```
Copy

Docker

Isolated container testing.

```
docker --version
```
Copy

VS Code

Best Python + Docker support

Hi! What can I help you with?

→

How to Submit?